

Mini Manual de Uso – MVC en PHP (Router propio)

Este documento describe de forma clara y práctica cómo utilizar el patrón **MVC (Modelo-Vista-Controlador)** en el proyecto PHP que estás desarrollando con Router propio.

1. Estructura General del Proyecto

```
/project-root
|
├── index.php           # Punto de entrada
├── Router.php          # Enrutador principal
|
├── config/
│   └── config.php      # Configuración general
|
├── controllers/
│   ├── AuthController.php
│   ├── HomeController.php
│   └── BaseController.php # (opcional)
|
├── models/
│   ├── Usuario.php
│   └── BaseModel.php    # (opcional)
|
├── views/
│   ├── layouts/
│   │   ├── header.php
│   │   ├── navbar.php
│   │   └── footer.php
│   ├── auth/
│   │   └── login.php
│   ├── home/
│   │   └── index.php
|
└── public/
    ├── css/
    ├── js/
    └── img/
```

2. Flujo MVC (Cómo funciona todo)

1. El navegador solicita una URL (`/login`)
 2. `index.php` recibe la petición
 3. El `Router` analiza la ruta
 4. Se ejecuta un **Controlador** y un **método**
 5. El controlador consulta el **Modelo** (si aplica)
 6. El controlador carga una **Vista**
 7. La vista se muestra al usuario
-

3. index.php (Punto de Entrada)

```
<?php
require 'config/config.php';
require 'Router.php';

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

$router = new Router();

$router->add('/', 'HomeController@index');
$router->add('/login', 'AuthController@index');
$router->add('/auth/login', 'AuthController@login');
$router->add('/logout', 'AuthController@logout');

$router->dispatch($_SERVER['REQUEST_URI']);
```

4. Rutas

Las rutas siguen el formato:

```
/ruta → Controlador@metodo
```

Ejemplo:

```
$router->add('/usuarios', 'UsuarioController@index');
```

5. Controladores

Los controladores contienen la **lógica de negocio**.

Ejemplo: AuthController

```
<?php
class AuthController
{
    public function index()
    {
        require 'views/auth/login.php';
    }

    public function login()
    {
        // Validar credenciales
        $_SESSION['usuario'] = 'admin';
        header('Location: /');
        exit;
    }

    public function logout()
    {
        session_destroy();
        header('Location: /login');
        exit;
    }
}
```

6. Modelos

Los modelos representan los **datos y acceso a base de datos**.

Ejemplo: Usuario.php

```
<?php
class Usuario
{
    public function autenticar($email, $password)
    {
        // Consulta a base de datos
        return true;
    }
}
```

```
}  
}
```

Buenas prácticas: - Un modelo por entidad - NO incluir HTML - NO manejar redirecciones

7. Vistas

Las vistas solo contienen **HTML + PHP mínimo**.

Ejemplo: views/auth/login.php

```
<?php require 'views/layouts/header.php'; ?>  
<?php require 'views/layouts/navbar.php'; ?>  
  
<form method="POST" action="/auth/login">  
    <input type="text" name="usuario" placeholder="Usuario">  
    <input type="password" name="password" placeholder="Contraseña">  
    <button type="submit">Ingresar</button>  
</form>  
  
<?php require 'views/layouts/footer.php'; ?>
```

8. Layouts y Parciales

Los layouts permiten reutilizar código.

- header.php → `<head>` y CSS
- navbar.php → menú de navegación
- footer.php → scripts JS

Esto evita duplicar código.

9. Sesiones y Protección de Rutas

```
if (!isset($_SESSION['usuario'])) {  
    header('Location: /login');  
    exit;  
}
```

Colocar al inicio de controladores protegidos.

10. Convenciones Recomendadas

- Controladores en **PascalCase**
- Métodos en **camelCase**
- Vistas en carpetas por módulo
- No usar lógica compleja en vistas
- Usar `exit` después de `header()`

11. Resumen

Componente	Responsabilidad
Router	Decide qué controlador usar
Controlador	Lógica de negocio
Modelo	Datos y BD
Vista	Interfaz de usuario

Este mini manual cubre el uso completo del MVC base del proyecto. Puede extenderse con middleware, validaciones, ORM o seguridad avanzada según sea necesario.

11. Permisos por Rol (Autorización)

Para controlar el acceso a funcionalidades según el rol del usuario, se implementa un **middleware simple de autorización**.

11.1 Middleware de Autenticación y Roles

Ruta: `app/helpers/AuthMiddleware.php`

```
<?php

class AuthMiddleware
{
    public static function auth()
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }
}
```

```

        if (!isset($_SESSION['usuario'])) {
            header("Location: /login");
            exit;
        }
    }

    public static function role($rolesPermitidos = [])
    {
        self::auth();

        if (!in_array($_SESSION['usuario']['rol'], $rolesPermitidos)) {
            header("HTTP/1.1 403 Forbidden");
            echo "Acceso denegado";
            exit;
        }
    }
}

```

11.2 Uso en Controladores

Solo autenticado:

```
AuthMiddleware::auth();
```

Por rol:

```
AuthMiddleware::role(['ADMIN']);
```

12. Logout con Limpieza Selectiva

El cierre de sesión se maneja desde el `AuthController`, eliminando únicamente la información del usuario.

12.1 Método `logout()`

```

public function logout()
{
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }
}

```

```

unset($_SESSION['usuario']);
session_regenerate_id(true);

header("Location: /login");
exit;
}

```

13. Bitácora de Accesos

Se registra cada inicio y cierre de sesión para fines de auditoría.

13.1 Tabla de Bitácora

```

CREATE TABLE bitacora_accesos (
    idBitacora INT AUTO_INCREMENT PRIMARY KEY,
    idUsuario INT,
    accion ENUM('LOGIN','LOGOUT') NOT NULL,
    ip VARCHAR(45),
    userAgent VARCHAR(255),
    fecha DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

13.2 Modelo Bitacora

Ruta: `app/models/Bitacora.php`

```

<?php

class Bitacora
{
    public function registrar($idUsuario, $accion)
    {
        $db = db();

        $stmt = $db->prepare(
            "INSERT INTO bitacora_accesos (idUsuario, accion, ip, userAgent)
VALUES (?, ?, ?, ?)"
        );

        $ip = $_SERVER['REMOTE_ADDR'] ?? 'N/A';
        $ua = $_SERVER['HTTP_USER_AGENT'] ?? 'N/A';

        $stmt->bind_param("iss", $idUsuario, $accion, $ip, $ua);
    }
}

```

```
        $stmt->execute();  
    }  
}
```

13.3 Registro en Login y Logout

Login:

```
$bitacora = new Bitacora();  
$bitacora->registrar($_SESSION['usuario']['idUsuario'], 'LOGIN');
```

Logout:

```
$bitacora = new Bitacora();  
$bitacora->registrar($_SESSION['usuario']['idUsuario'], 'LOGOUT');
```

14. Flujo Final de Autenticación y Seguridad

1. Usuario inicia sesión
2. Se validan credenciales
3. Se consulta `vw_usuarioPersona`
4. Se crea sesión de usuario
5. Se registra LOGIN en bitácora
6. Se valida acceso por rol
7. Usuario cierra sesión
8. Se registra LOGOUT en bitácora
9. Se limpia sesión selectivamente

Este esquema proporciona una base **institucional, segura y escalable** para el control de acceso dentro del sistema MVC.